

Quizmaster

Arhitektura projekta

Kosta Arsić 18565
Marko Pavlović 18996

1. Kontekst i cilj softverskog projekta	3
2. Projektni zahtevi	3
2.1. Pregled ključnih ciljeva	3
2.2. Slučajevi korišćenja	3
2.3. Ograničenja	3
2.4. Funkcionalni zahtevi	3
2.5. Nefunkcionalni zahtevi	4
3. Arhitekturni dizajn	4
3.1. Relevantni arhitekturni obrazci	4
3.1.1. 2-Layer Client Server	4
3.1.2. Publish-Subscribe	4
3.1.3. Layered	4
3.1.4. Repository	5
3.2. Pregled arhitekture	5
3.3. Strukturni pogled	6
3.4. Fizički pogled	7
3.5. Bihevioralni pogled	8
3.5.1. Login	8
3.5.2. Pribavljanje lobby-a za igru	9
3.5.3. Ulazak u lobby	10
3.5.4. Odgovor na pitanje	11
3.6. Klasni pogled (backend)	12
4. Tehnologije implementacije	12
5. Model podataka	13
5.1. User	13
5.2. Quiz	14
5.3. QuizQuestion	14
5.4. Lobby	14
5.5. LobbySession	15
5.6. LobbyLog	15
6. Funkcionalnost kviza	15

1. Kontekst i cilj softverskog projekta

Trenutno, na tržištu nema izobilja platformi na kojima korisnici mogu jednostavno i pristupačno da održavaju online kvizove. Postoji znatna količina kviz igara sa postojećim, fiksnim setovima pitanja, ali je retko da dozvoljavaju korisnicima da kreiraju sopstvene setove pitanja, i da održavaju svoj kviz preko interneta.

Projekat "Quizmaster" za cilj ima kreiranje online platforme za kreiranje i održavanje kvizova sa prijateljima, porodicom ili čak i za edukativne potrebe.

2. Projektni zahtevi

2.1. Pregled ključnih ciljeva

- Omogućiti korisnicima lak pristup online kvizovima u pravom vremenu
- Omogućiti korisnicima da sami kreiraju svoje kvizove proizvoljnog sadržaja i teme u kojima zatim mogu učestvovati drugi korisnici
- Podržati kolaborativno (u timu) i konkurentno učešće u kvizovima

2.2. Slučajevi korišćenja

- Lična zabava u slobodno vreme - samostalno, sa prijateljima, ili sa porodicom
- Samostalna edukacija kroz kvizove o željenim temama
- Takmičenje u znanju sa drugim korisnicima
- Održavanje kvizova u učionici za edukativne svrhe

2.3. Ograničenja

- Korisnici će imati pristup aplikaciji samo iz Srbije
- Dozvoljeni format pitanja u kvizovima je izbor jedne od 3 ili 4 opcije
- Radi limitiranja opterećenja servera broj učesnika u jednom kvizu je maksimalno 4.

2.4. Funkcionalni zahtevi

- Istovremeno učestvovanje više korisnika u kvizu preko interneta
- Sposobnost servera da održava veći broj aktivnih kvizova u jednom trenutku

- Mogućnost organizovanja kviza i formiranja "čekaonice" kojoj se drugi korisnici mogu pridružiti
- Pristup svim postojećim kvizovima u bazi od strane svih korisnika
- Interfejs za kreiranje novih kvizova dostupan svim korisnicima
- Pristup aplikaciji kroz kreiranje profila i logovanje u profil
- Mogućnost pristupa aplikaciji sa različitih uređaja (računar, laptop, telefon), operativnih sistema (Windows, Linux, MacOS), i web pretraživača (Chorme, Firefox, Edge)

2.5. Nefunkcionalni zahtevi

- Sve promene stanja u toku izrade kviza se prosleđuju svim učesnicima istovremeno
- Brzo i lako organizovanje i pokretanje kviza bez značajnog čekanja
- Prost interfejs za izvršavanje svih funkcionalnosti aplikacije
- Eliminisanje mogućnosti neželjenog prekida kviza u toku njegovog održavanja
- Mogućnost eventualne buduće izmene, nadogradnje, ili proširenja sistema

3. Arhitekturni dizajn

3.1. Relevantni arhitekturni obrazci

3.1.1. 2-Layer Client Server

Za potrebe ovog projekta je potreban relativno prost dvoslojni client-server sistem, koji obezbeđuje komunikaciju i distribuciju podataka izmedju svih učesnika u kvizu.

3.1.2. Publish-Subscribe

Server implementira Publish-Subscribe model za obaveštavanje korisnika u toku igre o akcijama ostalih učesnika, radi sinhronizacije stanja igre i blokiranje aktivnosti nakon sto je jedan od igrača odlučio da odgovori na pitanje.

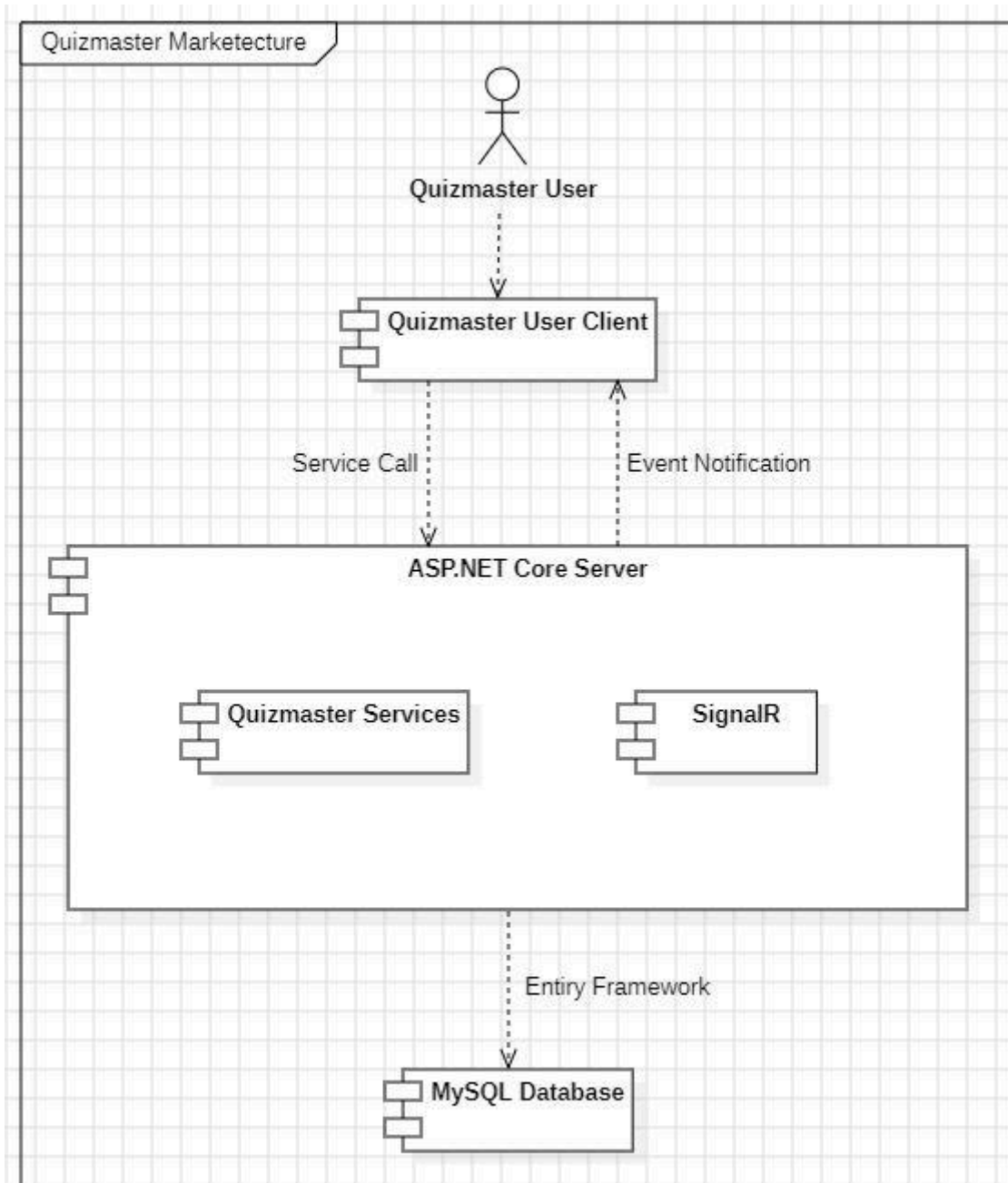
3.1.3. Layered

Server je organizovan interno u slojeve radi lakše raspodele dužnosti izmedju serverskih komponenata - primanje i parsovanje zahteva, obradu zahteva i model podataka na serveru.

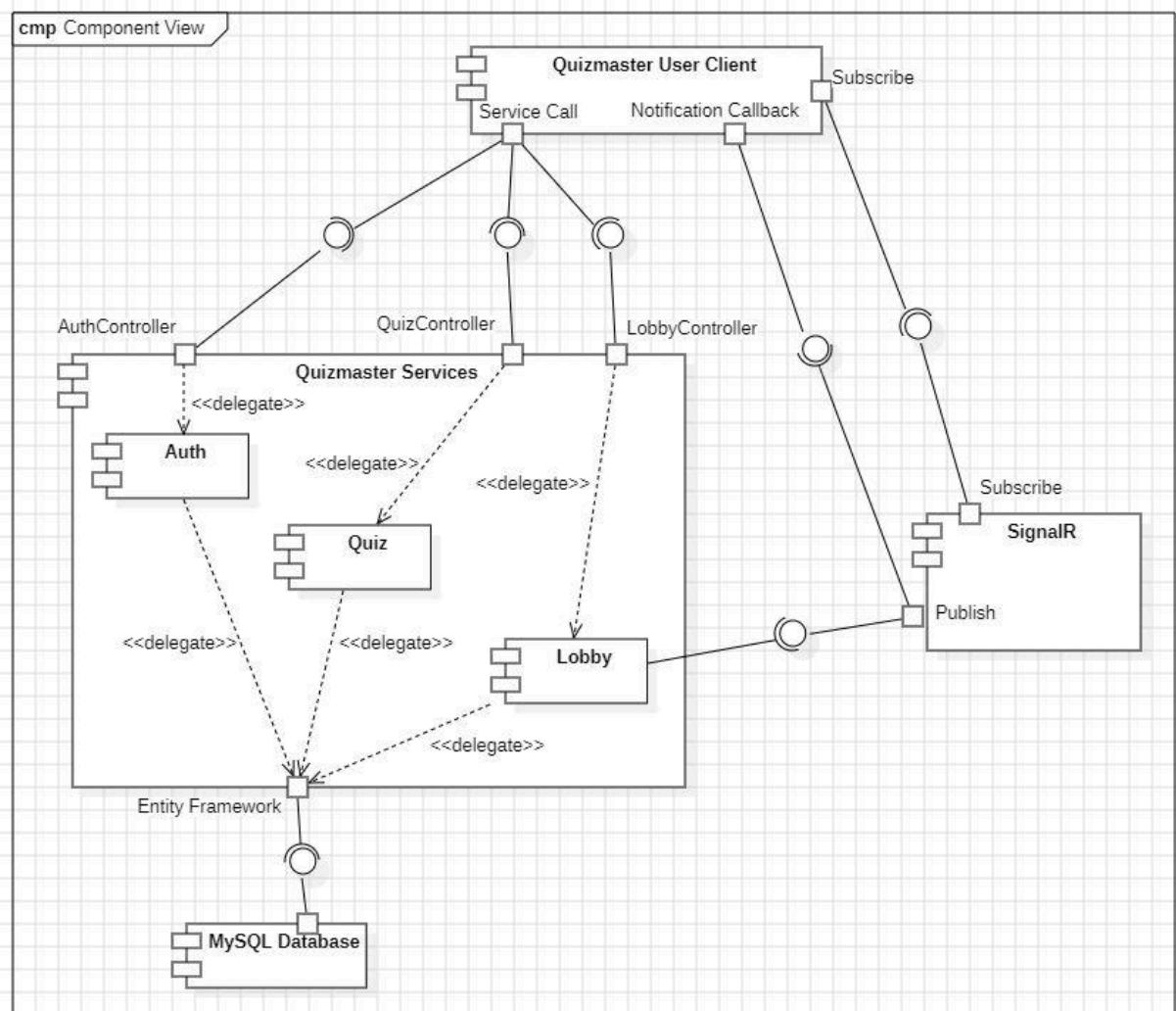
3.1.4. Repository

Baza podataka služi za efikasnu perzistenciju podataka o korisniku, paketima kvizova i pojedinačnim pitanjima. Ovim podacima se kasnije pristupa u toku samog kviza.

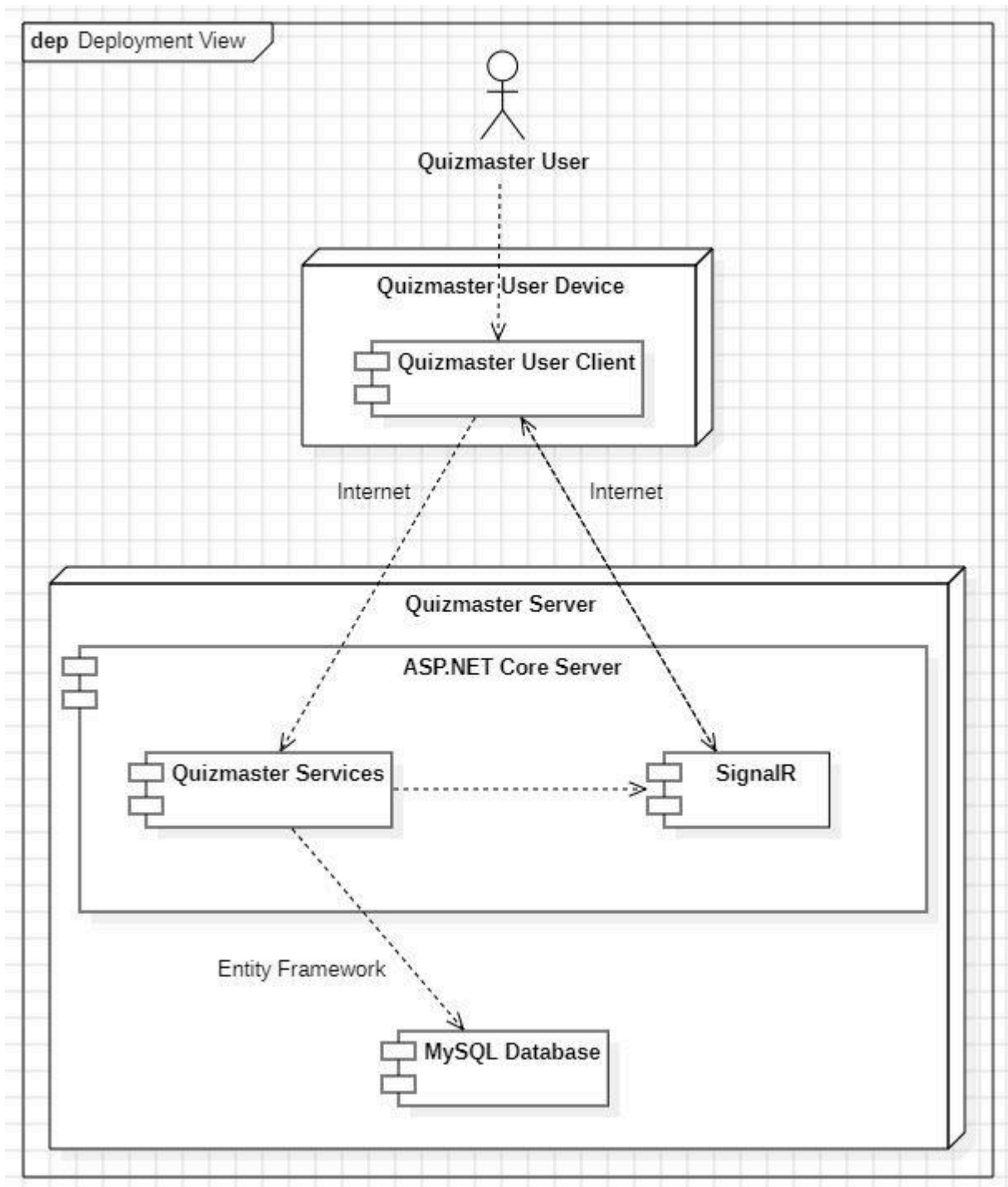
3.2. Pregled arhitekture



3.3. Strukturni pogled

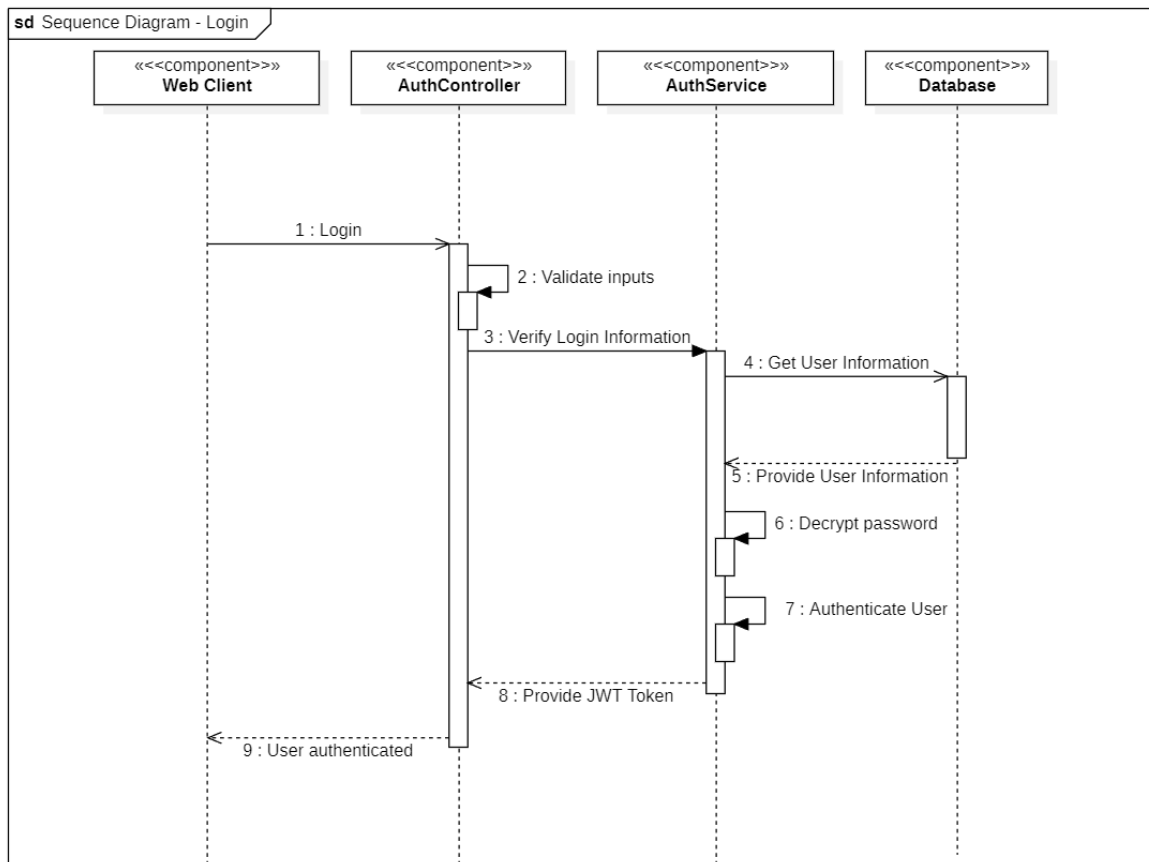


3.4. Fizički pogled

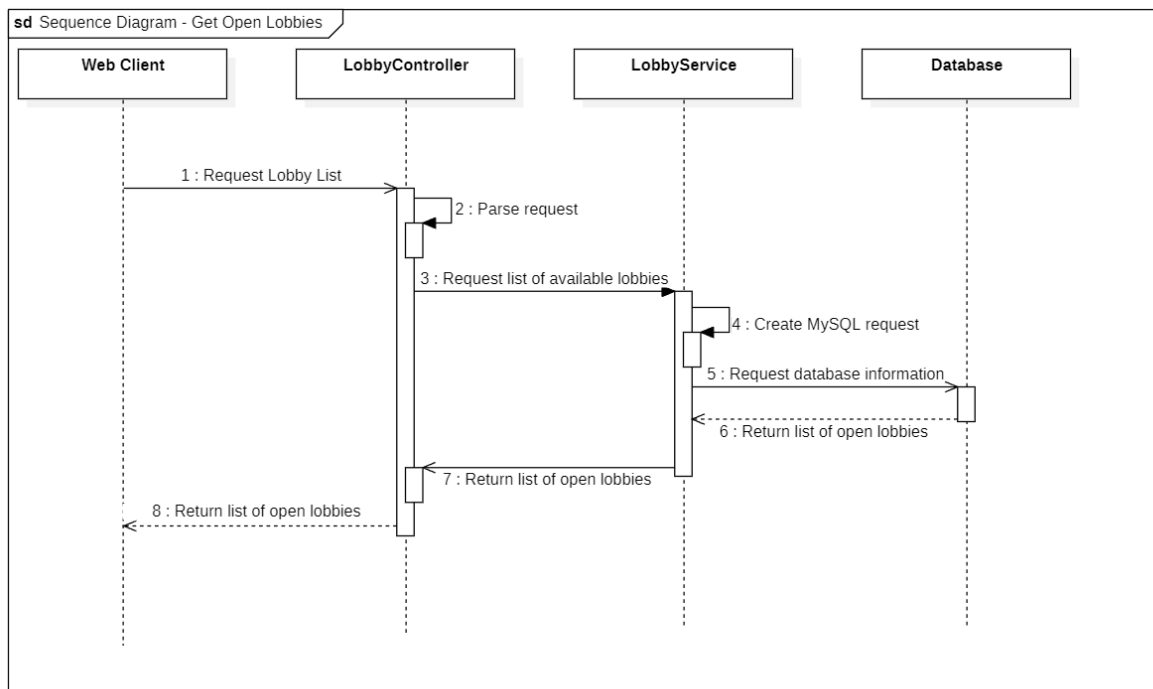


3.5. Bihevioralni pogled

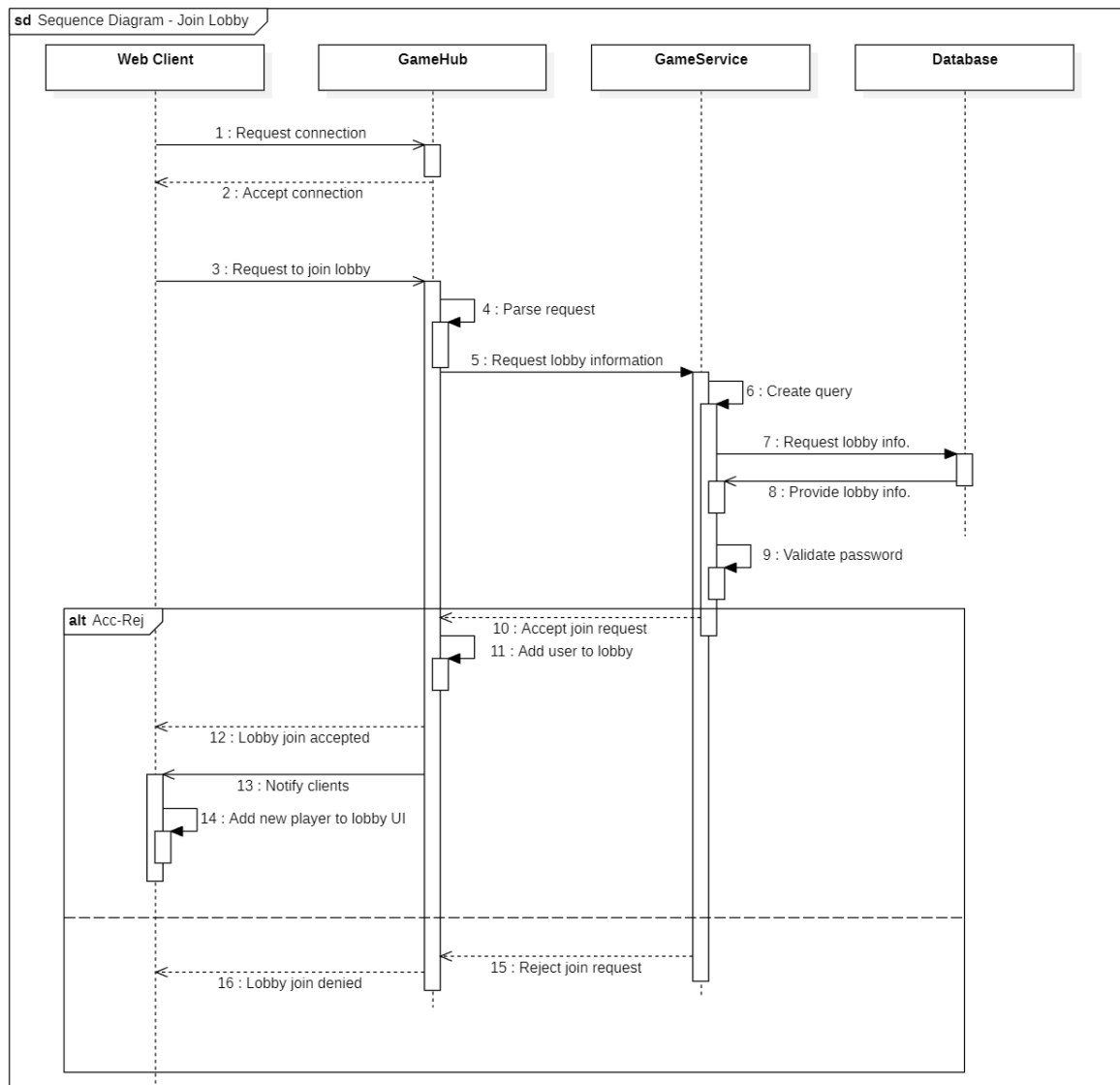
3.5.1. Login



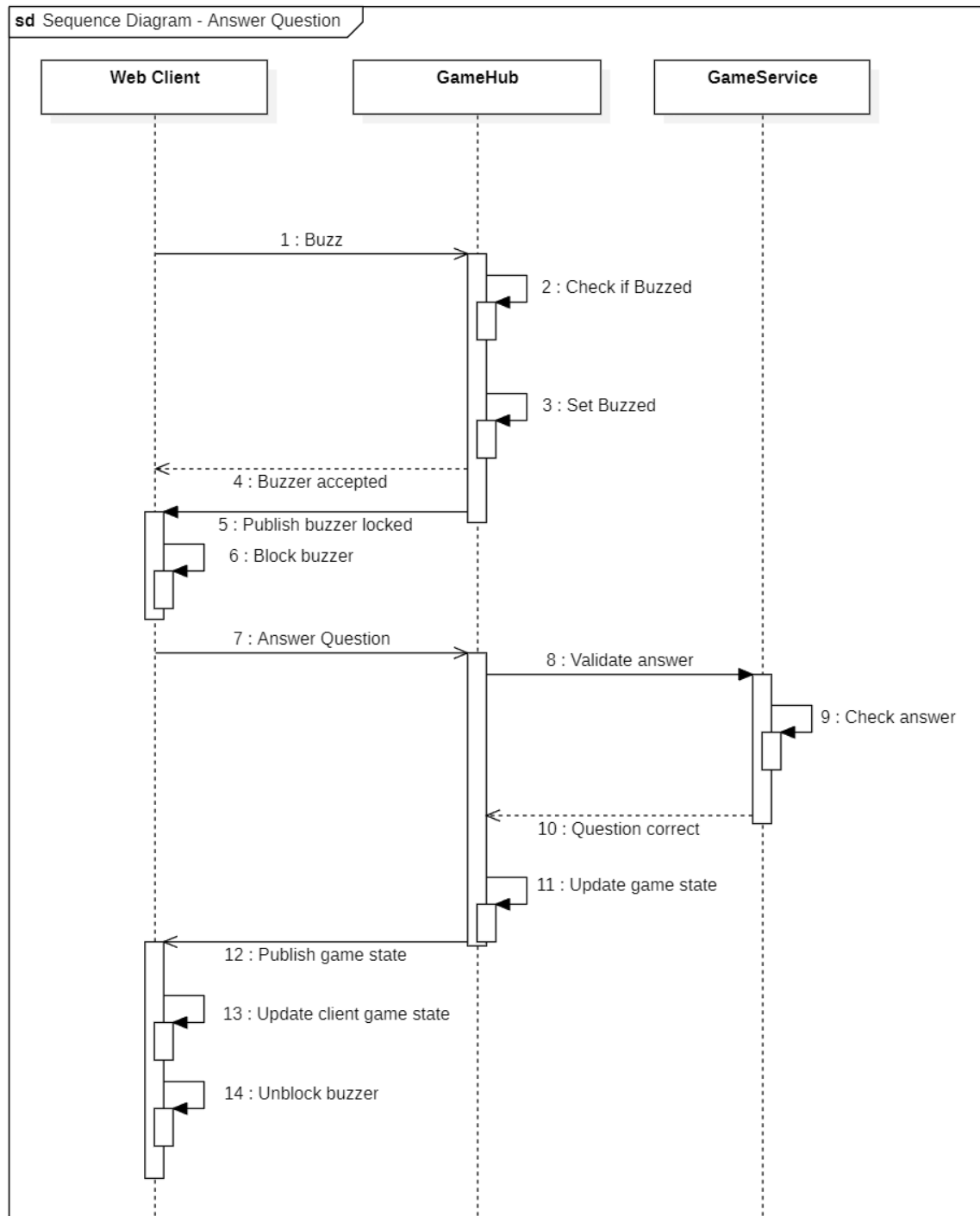
3.5.2. Pribavljanje lobby-a za igru



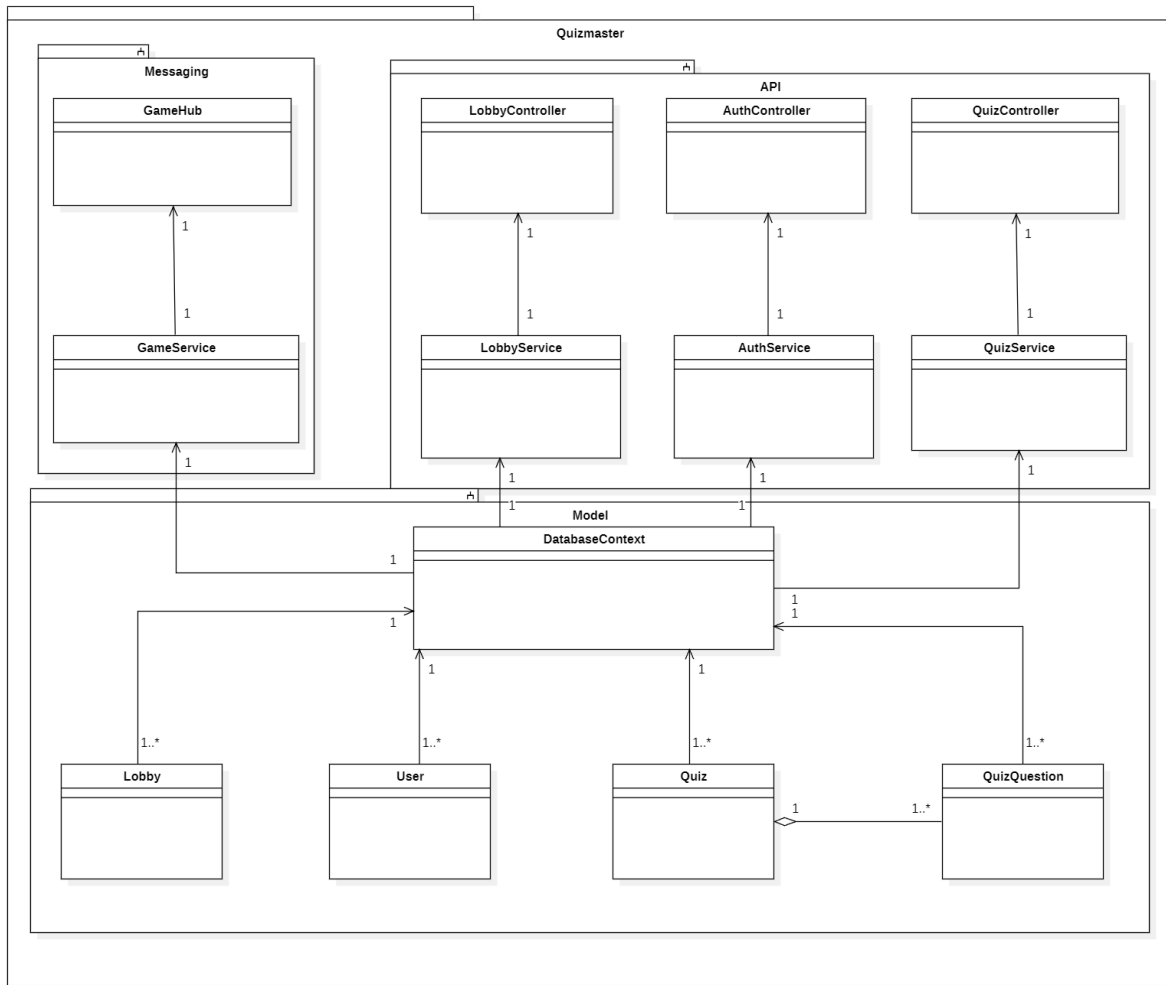
3.5.3. Ulazak u lobby



3.5.4. Odgovor na pitanje



3.6. Klasni pogled (backend)



4. Tehnologije implementacije

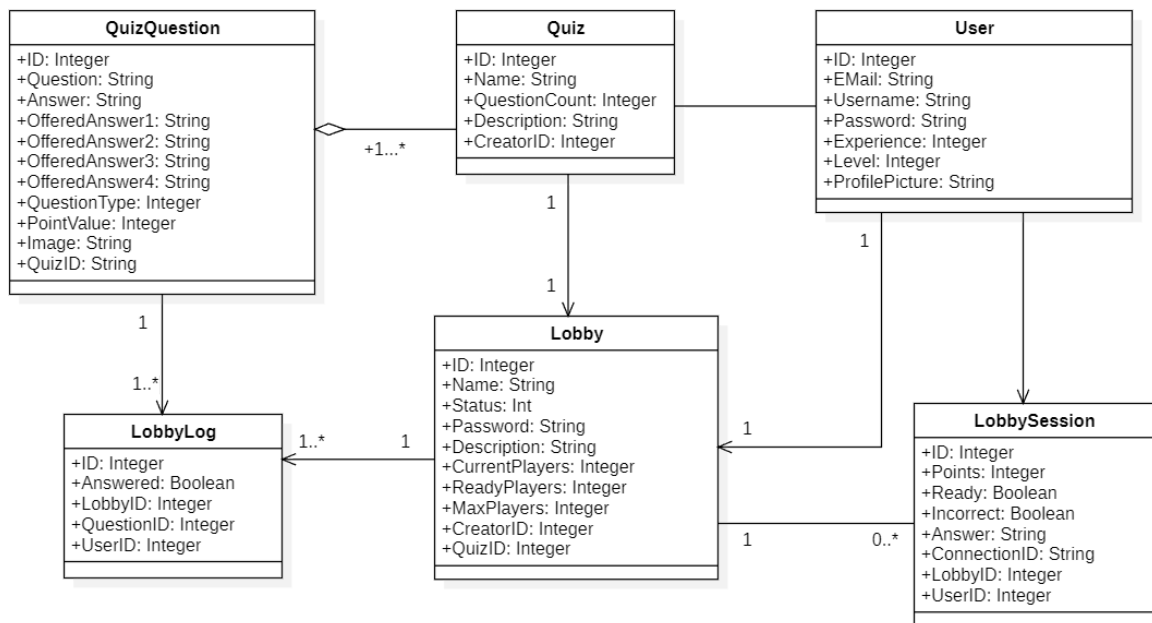
Korisnički interfejs web aplikacije će biti implementirana koristeći React biblioteku za Javascript, koja će funkcionisati na NodeJS live serveru.

Primanje i obradu korisničkih zahteva će biti implementirana koristeći C# ASP.Net Core Framework za kreiranje web aplikacija.

Perzistencija podataka će biti implementirana koristeći MySQL relacioni DBMS.

Komunikacija izmedju korisnika će biti implementirana koristeći SignalR biblioteke za React i [ASP.Net](#).

5. Model podataka



Model podataka aplikacije Quizmaster se zasniva na modelu od šest klasa kojim se modeluju potrebni objekti za funkcionalnost aplikacije. Ovih šest klasa su: User, Lobby i njegove komponente - LobbyLog i LobbySession, Quiz i njegova komponenta QuizSession. Podaci u ovom obliku se koriste u svakom od tri sloja aplikacije, modelovani su ovako u bazi podataka, business-sloju za obradu kviza i front-end prikazu kviza, što olakšava komunikaciju izmedju slojeva.

5.1. User

User je klasa koja predstavlja sve korisnike aplikacije. Njegova uloga u aplikaciji je za čuvanje podataka o različitim korisnicima, i olakšavanje komunikacije izmedju korisnika tokom samog kviza. Atributi ove klase su:

- ID: Identifikacioni broj.
- E-Mail - E-Mail adresa korisnika, korišćena pri registraciji.
- Password: Heširana šifra korisnika.
- Experience i Level: Vrednosti korišćene za predstavljanje iskustva korisnika.
- ProfilePicture: Čuva ime profilne slike.

5.2. Quiz

Quiz je klasa koja predstavlja celokupan skup pitanja koja se koriste tokom kviza. Njegovi atributi su:

- **ID**: Identifikacioni broj kviza
- **Name**: Ime kviza.
- **QuestionCount**: Broj pitanja koje kviz sadrži.
- **Description**: Kratak opis kviza.
- **CreatorID**: Identifikacioni broj korisnika koji je kreirao kviz, koji je referenca na User.

5.3. QuizQuestion

QuizQuestion je klasa koja predstavlja individualno pitanje koje se nalazi u kvizu. Ova klasa je modularna, i u programu funkcioniše različito u zavisnosti od tipa pitanja. Atributi QuizQuestion su:

- **ID**: Identifikacioni broj pitanja.
- **Question**: Tekst pitanja.
- **Answer**: Tačan odgovor na pitanje.
- **OfferedAnswer1 - 4**: U zavisnosti na tip pitanja, ovi slotovi se koriste kao ponudjeni izbori korisniku.
- **QuestionType**: Integer koji predstavlja tip pitanja.
- **PointValue**: Vrednost pitanja u bodovima koji se broje tokom kviza.
- **Image**: Slika asocirana sa pitanjem, ako to tip pitanja dozvoljava.
- **QuizID**: Referenca na kviz u kojem se ovo pitanje nalazi.

5.4. Lobby

Lobby je klasa koja predstavlja sobe u koje ulaze korisnici da bi učestvovali u kvizu. Skoro sve druge komponente referenciraju Lobby na neki način. Atributi Lobby su:

- **ID**: Identifikacioni broj sobe.
- **Name**: Ime sobe.
- **Status**: Predstavlja da li je soba otvorena, zatvorena ili trenutno u procesu igre.
- **Password**: Šifra potrebna da se udje u sobu.
- **Description**: Opis sobe.
- **CurrentPlayers**: Trenutan broj igrača.
- **MaxPlayers**: Maksimalan broj igrača.
- **ReadyPlayers**: Broj igrača pripremljen za početak igre.
- **CreatorID**: Identifikacioni broj kreatora sobe, koji može da modifikuje attribute sobe i da započne kviz.
- **QuizID**: Identifikacioni broj kviza koji je izabran za ovu sobu.

5.5. LobbySession

LobbySession je sastavni deo Lobby sistema, koji predstavlja individualnu konekciju koju korisnik ima sa sobom u koju je ušao. Ova klasa se koristi i za čuvanje stranja korisnika u datoj sobi. Njeni atributi su:

- ID: Identifikacioni broj sesije.
- Points: Broj poena koji je korisnik zaradio u sesiji.
- Ready: Atribut koji čuva spremnost korisnika za početak kviza.
- Incorrect: Ako je korisnik pogrešio trenutno pitanje, ovaj atribut čuva tu činjenicu i sprečava da pokuša ponovo da odgovori na njega.
- Answer: Poslednji odgovor koji je korisnik poslao - prikazuje se na ekranu u odredjenim situacijama.
- ConnectionID: ID konekcije - korišćen interno za komunikaciju.
- LobbyID: ID sobe za koju je sesija vezana.
- UserID: ID korisnika za koju je sesija vezana.

5.6. LobbyLog

LobbyLog služi za čuvanje stanja sobe tokom kviza - koja su pitanja odgovorena, i ko ih je odgovorio. Atributi su:

- ID: Identifikacioni broj log-a.
- QuestionID: Identifikacioni broj pitanja asociran za ovaj log.
- Answered: Da li je pitanje odgovoreno?
- UserID: ID korisnika koji je odgovorio na log.
- LobbyID: ID sobe asociranom sa sobom.

6. Komunikacija i Lobby sistem

GameHub
+dbContext: DbContext +cancellationTokens: Dictionary<>
+AddToLobby(Integer lobbyID, Integer userID) +RemoveFromLobby(Integer lobbyID, Integer userID) +Ready() +Unready() +NotifyOfLobbyUpdate(Integer lobbyID) +StartQuiz(Integer lobbyID, Integer quizID) +SendNextQuestion(Integer lobbyID) +StopTimer(Integer lobbyID) +ResumeQuiz(Integer lobbyID) +StopQuiz(Integer lobbyID) +BuzzIn(Integer lobbyID, Integer questionID, Integer userID) +AnswerQuestion(Integer lobbyID, Integer questionID, Integer userID, String answer) +Timeout(Integer lobbyID, Integer questionID, Integer seconds, CancellationToken cancellationToken) +OnDisconnectedAsync(Exception? exception)

Funkcionalnost izvođenja kviza nosi klasa GameHub. Ova klasa koristi SignalR - biblioteku koja implementira publish-subscribe model komunikacije gde se klijenti mogu prijaviti da dobijaju notifikacije od servera.

Komunikacija se osniva na principu SignalR grupa - skupova konekcija koje su pretplacene na određeni stream poruka. Ovo je korišćeno za implementaciju lobby sistema - Korisnici su samo obavesteni za promene u stanju sobe u kojoj se trenutno nalaze, i ni za koju drugu sobu.

6.1. Upravljanje sobom

Ulazanje i izlazanje iz sobe obradjuju funkcije GameHub-a:

- **AddToLobby**: Funkcija koja prima ID-eve korisnika i sobe u koju korisnik ulazi, i dodaje datog korisnika u grupu vezanu za tu sobu.
- **RemoveFromLobby**: Funkcija koja prima ID-eve korisnika i sobe iz koje korisnik izlazi, i uklanja datog korisnika iz grupe vezanu za tu sobu.
- **OnDisconnectedAsync**: U slučaju gubitka konekcije u toku kviza, ili u toku lobby-a, korisnik je uklonjen iz sobe automatski.
- **NotifyOfLobbyUpdate**: U slučaju bilo koje promene sobe, svaki korisnik koji je deo te sobe je obavešten o promeni, i naveden da preuzme nove podatke vezane za tu sobu.

6.2. Započinjanje igre

Zapocinjanje igre obradjuju funkcije GameHub-a:

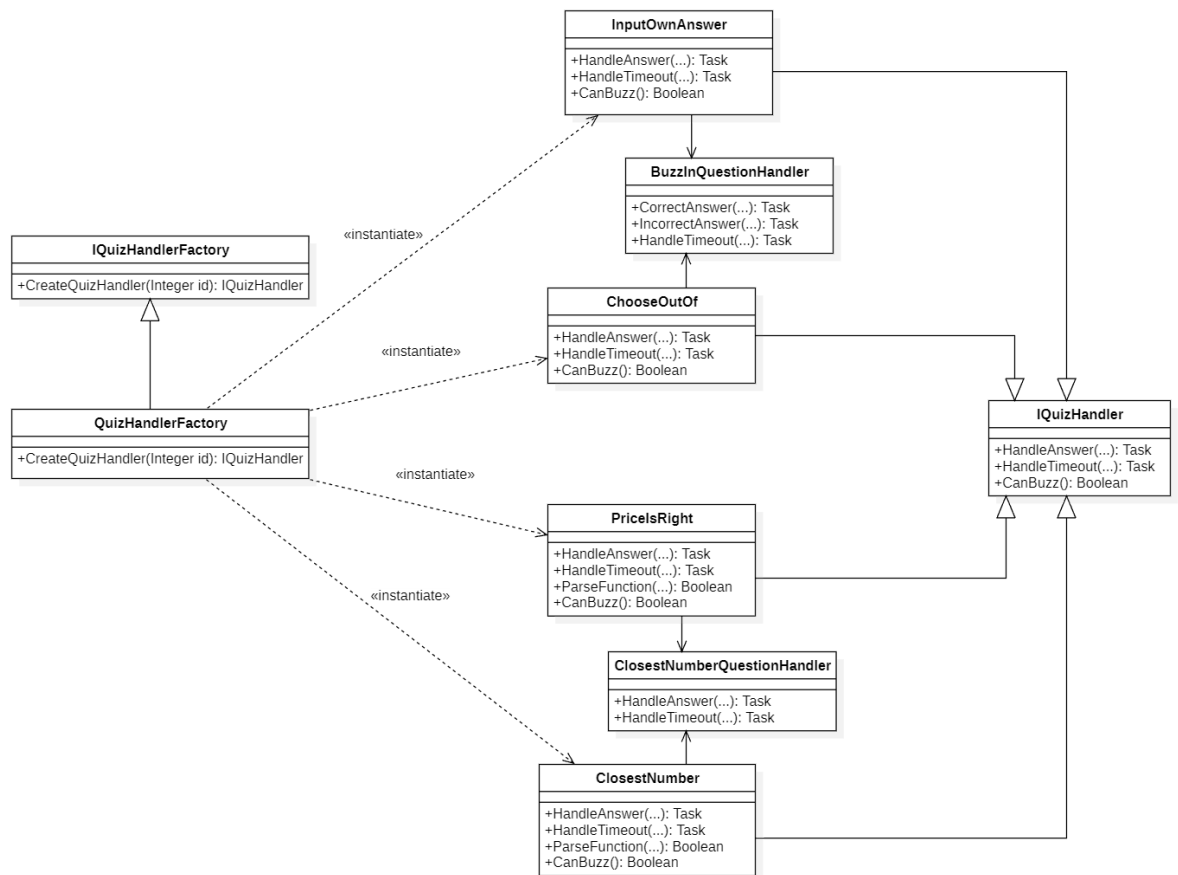
- **Ready:** Funkcija koja na osnovu ConnectionString-a postavlja da je korisnik spreman za pocetak igre. Ovo se smatra promenom u sobi, i poziva NotifyOfLobbyUpdate.
- **Unready:** Funkcija koja na osnovu ConnectionString-a postavlja da korisnik nije spreman za pocetak kviza. Ovo se smatra promenom u sobi, i poziva NotifyOfLobbyUpdate.
- **StartQuiz:** Ako nisu svi korisnici u sobi spremni, funkcija ne radi nista. Ako jesu, ova funkcija zapocinje proces kojim se dostavljaju pitanja klijentima, i obavestava ih o pocetku kviza.
- **EndQuiz:** U slucaju da nema vise pitanja, kviz se zavrшава pozivom EndQuiz, koji prosledjuje svim relevantnim korisnicima poruku da je kviz završen, i dodeljuje EXP bodove i levele.

6.3. Odgovaranje na pitanja - QuizHandler

Obradu odgovora na pitanja takodje vrse funkcije GameHub-a:

- **SendNextQuestion** - Prventstveno, ovo je funkcija koja nalazi sledece pitanje koje se treba izvorsiti, i salje ga ostalim korisnicima. Pored toga, funkcija je zaduzena za upravljanje stanja kvizom pre slanje sledeceg pitanja, i za slanje sledeceg pitanja. Uklanjaju se cancellation tokens, resetuju se stanja svih korisnika, poziva kraj kviza kad je potrebno.
- **BuzzIn** - Funkcija koja simulira "pritiskanje dugmeta" za odgovor na pitanje. Salje poruku koje zaustavlja bilo kojeg drugog korisnika da odgovori na pitanje, i daje pravo korisniku koji je pritisnuo dugme da odgovori.
- **AnswerQuestion** - Handler za odgovore na pitanja. Obradjuje odgovor na pitanje, u zavisnosti od tipa datog pitanja, i obavestava ostale korisnike.
- **Timeout** - U slucaju da korisnici ne odgovore na pitanje na vreme, poziva se Timeout funkcija koja obavestava korisnike, i poziva SendNextQuestion.

7. Obrada odgovora na pitanja



Obrada pitanja je relativno slozen sistem, kreiran za lako prosirivanje sistema. Zasnovan je na tri projektne obrasce: **Abstract Factory**, **Factory** i **Strategy**.

AbstractFactory implementira `IQuizHandlerFactory`. Ovaj interfejs služi za potencijalno instanciranje različitih tipova kvizova! U slučaju da je potrebno da se tipovi pitanja različito ponasaju za različite kvizove, može se implementirati različit `AbstractFactory` za kreiranje tog kviza.

Factory implementira `QuizHandlerFactory`. `QuizHandlerFactory` je zaslužan za instanciranje različitih `QuizHandler` u zavisnosti od tipa pitanja koji se trenutno obrađuje. Prima ID, koji predstavlja tip pitanja, i vraća odgovarajući `QuizHandler` objekat.

Ovo se nadovezuje na **Strategy** - Svi objekti koji kreira `Factory` su različite implementacija istog interfejsa - `IQuizHandler`, koji predstavlja različite algoritme koje se koriste za obradu pitanja.

Ti algoritmi su grupisani trenutno u dva tipa pitanja: "Buzz In" pitanja i "Closest Number" pitanja.

Buzz In kategorija obuhvata tipove pitanja koja koriste **BuzzInQuestionHandler** - **ChooseOutOf** i **InputOwnAnswer**.

Osnova ove kategorije je "Buzz In" dugme, koje dozvoljava korisniku da preuzme prioritet za odgovaranje pitanja. Svaki odgovor može biti isključivo tačan, što poziva **CorrectAnswer**, ili netačan, što poziva **IncorrectAnswer**. Odgovori korisnika ne zavise od odgovora drugih korisnika. Ostali korisnici ne mogu da odgovaraju na pitanje dok neko drugi ima prioritet.

HandleTimeout se aktivira samo kada ponestane vremena - i podrazumeva da je svako pogresio pitanje.

Closest Number kategorija obuhvata tipove pitanja koja koriste

ClosestNumberQuestionHandler, **ClosestNumber** i **PricelsRight**.

Osnova ove kategorije je činjenica da, pre nego što se pozove kraj pitanja, svi korisnici moraju da pošalju odgovor pre nego što se odgovor validira.

HandleAnswer samo obrađuje odgovore i čuva ih za obradu, koju vrši

HandleTimeout. **HandleTimeout** je pozvan kada više nema vremena, ili kada su svi korisnici odgovorili na pitanje. Vrsi se obrada svih pitanja, i u zavisnosti od toga koji i koliko je korisnika najbliže odgovoru, određuje ko je tačan, a ko netačno odgovorio na pitanje.

Tipovi pitanja su sledeći:

- **ChooseOutOf** - Korisnicima je prikazano pitanje i ponudjena 4 odgovora. Korisnik klikne na buzzer, i bira odgovor. Ako pogresi, ostalima se ponudi da odgovore. Ako pogodi, prikazuje se tačan odgovor i nastavlja se sa kvizom.
- **InputOwnAnswer** - Korisnicima je prikazano pitanje, i dato polje za unos. Ako pogresi, ostalima se ponudi da odgovore. Ako pogodi, prikazuje se tačan odgovor i nastavlja se sa kvizom.
- **ClosestNumber** - Korisnicima je prikazano pitanje, i dato polje za unos. Korisnik/Korisnici čiji je odgovor najbliži u vrednosti tačnom odgovoru se smatra da je tačno odgovorio.
- **PricelsRight** - Korisnicima je prikazano pitanje, i dato polje za unos. Korisnik/Korisnici čiji je odgovor najbliži u vrednosti tačnom odgovoru, *a da vrednost ne prelazi vrednost odgovora*, se smatra da je tačno odgovorio.